

LAPORAN PRAKTIKUM

Algoritma dan Pemrograman Komputer

Disusun untuk Memenuhi Mata Kuliah Algoritma dan Pemrograman Komputer

Dosen: Jevri Tri Ardiansah, S.Pd., M.Eng., Ph.D.



Disusun Oleh Kelompok 3 :

Ahmad Ibrohim Shofiyullah	260535646628
Anandhika Rizaldy	260535649775
Bintang Rachmad Hidayat	260535655667
Muhammad Zaki Aliyyudin	260535649406
Nadhira	260535654233
Nadila Zahra' Dzakkiyah	260535650089

PROGRAM STUDI S1 TEKNIK INFORMATIKA
DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS NEGERI MALANG
2026

KATA PENGANTAR

Puji syukur kami panjatkan kepada Tuhan Yang Maha Esa, karena atas kuasa dan kesempatannya lah kami dapat menyusun laporan ini sebagai bentuk penuntasan dari praktikum yang disusun untuk mata kuliah Algoritma dan Pemrograman. Dan atas kehendaknya lah laporan praktikum ini dapat terselesaikan dengan atau minim kendala.

Tak luput kami ucapkan kepada Bapak Jevri Tri Ardiansah selaku dosen dari mata kuliah Algoritma dan Pemrograman Komputer, karena atas kesempatan beliau lah kami dapat melaksanakan praktikum ini sebagai bentuk dari pemenuhan nilai dan sarana pengembangan pengetahuan Mahasiswa.

Terimakasih juga kami ucapkan kepada Kak Asyam Sakhi dan Kak Ainis Nur yang berperan sebagai Asisten Praktikum bagi mata kuliah Algoritma dan Pemrograman dari Offering TI-A. Karena dengan adanya kesempatan dan bantuan pengetahuan dari merekalah praktikum serta laporan ini dapat terlaksana dan diselesaikan. Semoga rahmat Tuhan senantiasa menyertai kehidupan mereka.

Terakhir, kami ucapkan terimakasih yang sebesar-besarnya terhadap seluruh personil kelompok dari penugasan laporan praktikum ini. Semoga tenaga, waktu, dan dedikasi yang diberikan dapat menjadi manfaat dan berkah bagi mereka yang turut andil dalam penyelesaian laporan praktikum ini.

DAFTAR ISI

KATA PENGANTAR	2
DAFTAR ISI.....	3
BAB I PENDAHULUAN	5
1.1 Latar Belakang	5
1.2 Rumusan Masalah	5
1.3 Tujuan	5
1.4 Manfaat	6
BAB II PEMBAHASAN	7
2.1 STUDI KASUS - KELAS.....	7
2.1.1 Kelas 1.....	7
2.1.2 Kelas 2.....	8
2.1.3 Kelas 3.....	9
2.1.4 Kelas 4.....	10
2.1.5 Kelas 5.....	11
2.2 STUDI KASUS - KELOMPOK	12
2.2.1 Studi Kasus 1 - HARGA DAN DISKON BARANG DI TOKO	13
2.2.2 Studi kasus 2 - ABSENSI MAHASISWA DALAM KELAS	16
2.2.3 Studi kasus 3 - PEMBAYARAN TAGIHAN LISTRIK	19
2.2.4 STUDI KASUS 4 - RATA-RATA NILAI DAN KATEGORI PRESTASI	22
2.2.5 STUDI KASUS 5 - PENGELOLAAN PENGELUARAN HARIAN	24

BAB III KESIMPULAN.....	28
3.1 Kesimpulan	28
LAMPIRAN Link Github	29

BAB I

PENDAHULUAN

1.1 Latar Belakang

Pemrograman komputer pada dasarnya adalah menyusun instruksi agar komputer dapat menyelesaikan suatu masalah secara otomatis. Dalam praktiknya, banyak pekerjaan yang harus dilakukan berulang kali, seperti menghitung total belanja dari beberapa barang, mencatat kehadiran setiap hari, atau mengolah nilai dari beberapa mata pelajaran. Menulis instruksi yang sama secara manual berkali-kali membuat program panjang, sulit dibaca, dan rawan kesalahan.

Untuk mengatasi hal tersebut, bahasa pemrograman menyediakan struktur kontrol perulangan (looping), yaitu mekanisme yang menjalankan sekumpulan pernyataan berulang kali selama kondisi tertentu terpenuhi. Dalam bahasa C++, perulangan dapat ditulis dengan pernyataan `while` dan `for`. Keduanya sering dipadukan dengan percabangan (`if-else`) agar program dapat mengambil keputusan, misalnya menentukan besar diskon atau kategori prestasi.

Materi ini termasuk dalam mata kuliah Algoritma dan Pemrograman, dengan capaian agar mahasiswa memahami struktur bahasa pemrograman dasar dan aturan penulisan C++, serta mampu menulis program C++ sederhana yang melibatkan input, proses, dan output. Karena itu, laporan ini membahas penerapan perulangan melalui lima studi kasus: harga dan diskon barang di toko, absensi mahasiswa, tagihan listrik, rata-rata nilai dan kategori prestasi, serta pengelolaan pengeluaran harian.

1.2 Rumusan Masalah

1. Bagaimana menerapkan perulangan yang dikombinasikan dengan percabangan untuk menyelesaikan kelima studi kasus?
2. Bagaimana menampilkan input, proses, dan output program dengan benar sesuai ketentuan tiap studi kasus?

1.3 Tujuan

1. Memahami konsep perulangan serta perbedaan penggunaan pernyataan while dan for pada C++.
2. Mampu membuat program C++ sederhana yang melibatkan input, proses, dan output dengan memanfaatkan perulangan dan percabangan.
3. Mampu menyelesaikan kelima studi kasus dan menyajikan hasil eksekusi programnya.

1.4 Manfaat

Manfaat yang kami dapatkan dari laporan praktikum ini ialah melatih logika pemrograman, ketelitian menulis sintaks C++, dan kemampuan menyelesaikan masalah nyata dengan program. Adapun bagi pembaca, para pembaca dapat menggunakan laporan praktikum ini sebagai referensi belajar penerapan perulangan dan percabangan pada C++ untuk kasus sehari-hari. Serta laporan praktikum ini menjadi bukti pemahaman materi looping dan bahan evaluasi capaian pembelajaran.

BAB II

PEMBAHASAN

2.1 STUDI KASUS - KELAS

2.1.1 Kelas 1

1) pemrograman

```
#include <iostream>
#include <iomanip>
using namespace std;
int main() {
    int sum = 0;
    for (int i = 1; i <= 5; i++) {
        sum += i;
        cout << "i: " << i << ", Sum: " << sum << endl;
    }
    cout << "Total: " << sum << endl;
}
```

- `iostream` dipakai untuk `cout` (menampilkan output).
- `iomanip` untuk format output (di program ini belum benar-benar dipakai).
- `using namespace std;` supaya tidak perlu menulis `std::cout`, cukup `cout`.
- Program dimulai dari `main()`. Variabel `sum` dibuat dengan nilai awal 0, berfungsi sebagai penampung total.
- `int i = 1` → mulai dari 1
- `i <= 5` → ulangi selama i kurang dari atau sama dengan 5
- `i++` → tambah i sebanyak 1 setiap putaran
- `sum += i` → sama dengan `sum = sum + i`, menambahkan nilai i ke sum
- `cout` → mencetak nilai i dan sum di setiap putaran

2) Output

```

PS C:\Users\Hp\Documents\tugas algoritma pemrograman\praktikum 4> cd "c:\Users\Hp\Documents\tugas algoritma pemrograman\p
raktikum 4\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
i: 1, Sum: 1
i: 2, Sum: 3
i: 3, Sum: 6
i: 4, Sum: 10
i: 5, Sum: 15
Total: 15
PS C:\Users\Hp\Documents\tugas algoritma pemrograman\praktikum 4> cd "c:\Users\Hp\Documents\tugas algoritma pemrograman\p
raktikum 4\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }

```

2.1.2 Kelas 2

1) Pemrograman

```

#include <iostream>

using namespace std;

int main() {
    int i = 10;
    for (i = 0; i < 10; i++)
    {
        cout << "i: " << i << endl;
    }
}

```

- `#include <iostream>` digunakan agar program dapat melakukan input dan output, terutama menggunakan `cout`.
- `using namespace std;` digunakan supaya tidak perlu menulis `std::` sebelum `cout` dan perintah lainnya.
- `int main() { }` merupakan fungsi utama yang pertama kali dijalankan ketika program dimulai.
- `int i = 10;` membuat variabel `i` dengan tipe data `int` dan memberikan nilai awal 10.
- `for (i = 0; i < 10; i++)` digunakan untuk melakukan perulangan.
- `i = 0` → menentukan nilai awal perulangan, yaitu 0.
- `i < 10` → menjadi kondisi perulangan. Perulangan akan terus berjalan selama nilai `i` masih kurang dari 10.
- `i++` → menambahkan nilai `i` sebanyak 1 setiap kali perulangan.
- `cout << "i: " << i << endl;` digunakan untuk menampilkan nilai `i` pada setiap perulangan.

2) Output

```
i: 0
i: 1
i: 2
i: 3
i: 4
i: 5
i: 6
i: 7
i: 8
i: 9
PS C:\Users\muhammad z a\AppData\Local\Temp> █
```

2.1.3 Kelas 3

1) Pemrograman

```
#include<iostream>
using namespace std;

int main () {

    int sum = 0;
    for (int i = 2; i <= 3; i++) {
        sum += i;
        cout << "Iterasi ke-" << i << ": Nilai i adalah " << i <<
        ", Jumlah saat ini adalah " << sum << endl; }
    cout << "Total penjumlahan: " << sum << endl;
    return 0;
}
```

#include<iostream>

Digunakan untuk memasukkan library iostream agar program dapat menggunakan **cout** dan **endl**.

using namespace std;

Memungkinkan penggunaan **cout** dan **endl** tanpa harus menulis **std::**.

int main() {

Menandakan bahwa program mulai dijalankan dari fungsi **main()**.

```
int sum = 0;
```

Membuat variabel `sum` dengan nilai awal 0. Variabel ini digunakan untuk menyimpan hasil penjumlahan.

```
for (int i = 2; i <= 3; i++) {
```

Membuat perulangan dengan aturan:

- `int i = 2` → nilai awal `i` adalah 2.
- `i <= 3` → perulangan berjalan selama `i` kurang dari atau sama dengan 3.
- `i++` → setiap selesai satu perulangan, nilai `i` bertambah 1.
- Jadi, perulangan terjadi untuk `i = 2` dan `i = 3`.

```
sum += i;
```

Menambahkan nilai `i` ke dalam `sum`.

Sama dengan `sum = sum + i`.

```
cout << "Iterasi ke-" << i << ...
```

Menampilkan nilai `i` dan jumlah sementara pada setiap perulangan.

```
cout << "Total penjumlahan: " << sum << endl;
```

Menampilkan hasil akhir penjumlahan setelah perulangan selesai.

2) Output

```
Iterasi ke-2: Nilai i adalah 2, Jumlah saat ini adalah 2
Iterasi ke-3: Nilai i adalah 3, Jumlah saat ini adalah 5
Total penjumlahan: 5
PS C:\Users\ACER\Documents\Algoritma dan Pemrograman\Praktikum 4>
```

2.1.4 Kelas 4

1) Pemrograman

```
#include<iostream>
using namespace std;

int main () {

    int i = 3;
```

```
for (; i <= 5; i++) {  
    cout << i << endl;  
}  
return 0;  
}
```

- `#include<iostream>`
Digunakan untuk menggunakan `cout` dan `endl`.
- `using namespace std;`
Agar kita bisa menulis `cout` dan `endl` tanpa `std::`.
- `int main() {`
Menandakan awal program.
- `int i = 3;`
Membuat variabel `i` dan memberikan nilai awal 3.
- `for (; i <= 5; i++) {`
Membuat perulangan `for`.
 - Bagian pertama kosong karena nilai awal `i` sudah diberikan sebelumnya yaitu 3.
 - `i <= 5` → perulangan berjalan selama `i` kurang dari atau sama dengan 5.
 - `i++` → setiap selesai satu perulangan, `i` bertambah 1.
- `cout << i << endl;`
Menampilkan nilai `i` ke layar.

2) Output

```
3  
4  
5  
PS C:\Users\ACER\Documents\Algoritma dan Pemrograman\Praktikum 4>
```

2.1.5 Kelas 5

1) Pemrograman

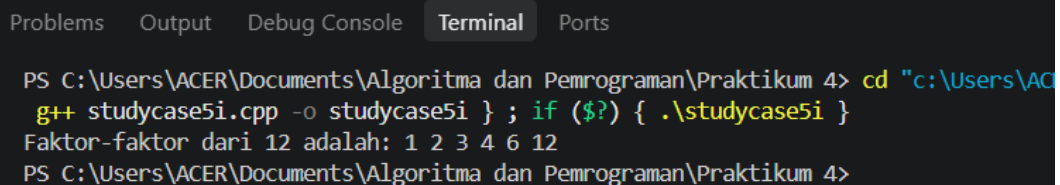
```
#include <iostream>
using namespace std;

int main() {
    int num;
    num = 12;

    cout << "Faktor-faktor dari " << num << " adalah: " ;
    for (int i = 1; i <= num; ++i) {
        if (num % i == 0) {
            cout << i << " ";
        }
    }
}
```

- `int num;`: Variabel `num` digunakan untuk menyimpan angka yang akan dicari faktor-faktornya.
- `num = 12;`: Disini `num` bernilai 12.
- `cout << "Faktor-faktor dari " << num;`: Menampilkan teks sebagai keterangan bahwa program akan mencari faktor-faktor dari angka 12.
- `for (int i = 1; i <= num; ++i)`: Perulangan `for` digunakan untuk memeriksa angka dari 1 sampai 12 satu per satu untuk mengetahui apakah angka tersebut merupakan faktor dari 12.
- `if (num % i == 0)`: Mengecek apakah `num` dapat dibagi habis oleh `i`. Operator `%` digunakan untuk mencari sisa pembagian. Jika sisanya 0, berarti `i` merupakan faktor dari `num`.
- `cout << i << " "`: Jika `i` merupakan faktor dari `num`, angka tersebut ditampilkan ke layar.

2) Output



```
Problems Output Debug Console Terminal Ports
PS C:\Users\ACER\Documents\Algoritma dan Pemrograman\Praktikum 4> cd "c:\Users\ACER\Documents\Algoritma dan Pemrograman\Praktikum 4" & g++ studycase5i.cpp -o studycase5i } ; if ($?) { .\studycase5i }
Faktor-faktor dari 12 adalah: 1 2 3 4 6 12
PS C:\Users\ACER\Documents\Algoritma dan Pemrograman\Praktikum 4>
```

2.2 STUDI KASUS - KELOMPOK

2.2.1 Studi Kasus 1 - HARGA DAN DISKON BARANG DI TOKO

1. Pemrograman

```
#include <iostream>
#include <iomanip>

using namespace std;

int main() {
//Dklarasi
    int jumlahBarang, tawaranBerulang;
    double hargaBarang, totalHarga = 0, diskonBarang = 0,
totalSetelahDiskon = 0;

    //Awalan
    do {
        cout << "Masukkan jumlah barang: ";
        cin >> jumlahBarang;

        totalHarga = 0;

        //Perulangan
        for (int i = 0; i < jumlahBarang; i++) {
            cout << "Masukkan harga barang ke-" << i + 1 <<
": Rp ";

            cin >> hargaBarang;

            totalHarga += hargaBarang;
        }

        //Diskon pertama
        if (totalHarga > 500000) {
            diskonBarang = totalHarga * 0.1;
        }
        //Diskon kedua
        else if (totalHarga >= 250000 && totalHarga <=
500000) {
            diskonBarang = totalHarga * 0.05;
        }
        //Tidak ada diskon
        else {
```

```

        diskonBarang = 0;
    }

    //Hitungan
    totalSetelahDiskon = totalHarga - diskonBarang;
    cout << fixed << setprecision(2);

    //Output
    cout << "Total Harga: " << totalHarga << endl;
    cout << "Diskon: " << diskonBarang << endl;
    cout << "Total Setelah Diskon: " <<
totalSetelahDiskon << endl;
        cout << "Ingin menambahkan belanjaan lagi? (1 untuk
ya, selain itu untuk tidak): ";
        cin >> tawaranBerulang;
    } while (tawaranBerulang == 1);

    return 0;
}

```

- `#include <iostream>` agar dapat I/O dan `#include <iomanip>` untuk menampilkan angka
- `using namespace std;` supaya tidak perlu menulis `std::` banyak
- `int main() { }` adalah fungsi utama pertama kali program dijalankan
- `int jumlahBarang, tawaranBerulang;` menggunakan `int` karena jumlah dan tawaran tidak boleh koma
- `double hargaBarang, totalHarga = 0, diskonBarang = 0, totalSetelahDiskon = 0;` berbeda dengan `double` yang bisa desimal sesuai dengan harga, diskon, dan total setelah diskon
- `do while` menandakan, operasikan `do` apabila (`while`) memiliki kondisi benar, menggunakan `syntax` ini bertujuan agar sistem dapat melakukan perulangan
- apabila (`if`) total harga mengikuti syarat diskon, harga barang akan dihitung sesuai dengan rumus diskon, maka pengoperasian berikut akan dijalankan
 - `if (totalHarga > 500000) {`
 - `diskonBarang = totalHarga * 0.1;`
 - `}`
 - `//Diskon kedua`

- else if (totalHarga >= 250000 && totalHarga <= 500000) {
- diskonBarang = totalHarga * 0.05;
- }
- //Tidak ada diskon
- else {
- diskonBarang = 0;
- Rumus diskon dalam pengodingan dapat ditulis totalSetelahDiskon = totalHarga - diskonBarang;
- cout << fixed << setprecision(2); agar double menghitung dua angka saja di belakang koma
- Keluaran yang dapat dilihat pengguna adalah
 - cout << "Total Harga: " << totalHarga << endl;
 - cout << "Diskon: " << diskonBarang << endl;
 - cout << "Total Setelah Diskon: " << totalSetelahDiskon << endl;
 - cout << "Ingin menambahkan belanjaan lagi? (1 untuk ya, selain itu untuk tidak): ";
 - cin >> tawaranBerulang;
- } while (tawaranBerulang == 1); sebagai syarat kondisi do di atas dapat dioperasikan
- return 0; sebagai penutup yang bersifat opsional. menandakan bahwa pemrograman tidak ada kesalahan

2. Output

```
Problems Output Debug Console Terminal Ports
PS C:\Users\USER> cd "c:\C++\" ; if ($?) { g++ coba.cpp -o coba } ; if ($?) { .\coba }
Masukkan jumlah barang: 3
Masukkan harga barang ke-1: Rp 50000
Masukkan harga barang ke-2: Rp 200000
Masukkan harga barang ke-3: Rp 400000
Total Harga: 650000.00
Diskon: 65000.00
Total Setelah Diskon: 585000.00
Ingin menambahkan belanjaan lagi? (1 untuk ya, selain itu untuk tidak): 1
Masukkan jumlah barang: 2
Masukkan harga barang ke-1: Rp 200000
Masukkan harga barang ke-2: Rp 100000
Total Harga: 300000.00
Diskon: 15000.00
Total Setelah Diskon: 285000.00
Ingin menambahkan belanjaan lagi? (1 untuk ya, selain itu untuk tidak): 1
Masukkan jumlah barang: 1
Masukkan harga barang ke-1: Rp 50000
Total Harga: 50000.00
Diskon: 0.00
Total Setelah Diskon: 50000.00
Ingin menambahkan belanjaan lagi? (1 untuk ya, selain itu untuk tidak): 2
PS C:\C++> █
```

2.2.2 Studi kasus 2 - ABSENSI MAHASISWA DALAM KELAS

1) pemrograman

```

1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      int lanjut = 1;
6
7      while (lanjut == 1) {
8          int totalHadir = 0;
9          int hadir;
10
11         for (int hari = 1; hari <= 5; hari++) {
12             cout << "Apakah mahasiswa hadir di hari ke-" << hari
13                 << "? (1 untuk hadir, 0 untuk tidak hadir): ";
14             cin >> hadir;
15
16             totalHadir += hadir;
17         }
18
19         int persentase = (totalHadir * 100) / 5;
20
21         cout << "Persentase Kehadiran: " << persentase << "%" << endl;
22
23         if (persentase > 75) {
24             cout << "Status Kehadiran: Baik" << endl;
25         }
26         else if (persentase >= 50) {
27             cout << "Status Kehadiran: Cukup" << endl;
28         }
29         else {
30             cout << "Status Kehadiran: Kurang" << endl;
31         }
32
33         cout << "Ingin mengecek kehadiran untuk mahasiswa lain? "
34             << "(1 untuk ya, selain itu untuk tidak): ";
35         cin >> lanjut;
36     }
37
38     return 0;
39 }

```

#include <iostream> agar program dapat melakukan input dan output, seperti menggunakan cin dan cout.

- using namespace std; supaya tidak perlu menulis std:: berulang kali pada cin, cout, dan fungsi lainnya.
- int main() { } merupakan fungsi utama dan menjadi bagian pertama yang dijalankan ketika program dimulai.

- `int lanjut = 1;` menggunakan tipe data `int` karena variabel `lanjut` digunakan untuk menyimpan angka 1 sebagai tanda untuk melanjutkan pengecekan mahasiswa.
- `while (lanjut == 1) { }` digunakan untuk melakukan perulangan selama nilai `lanjut` sama dengan 1.
- `int totalHadir = 0, hadir;` digunakan untuk menyimpan jumlah kehadiran mahasiswa. `totalHadir` dimulai dari 0, sedangkan `hadir` digunakan untuk menyimpan input kehadiran setiap hari.
- `for (int hari = 1; hari <= 5; hari++) { }` digunakan untuk melakukan perulangan sebanyak 5 kali, sesuai dengan jumlah hari dalam satu minggu yang ditentukan pada soal.
- `cout << ...` digunakan untuk menampilkan pertanyaan kepada pengguna mengenai kehadiran mahasiswa pada setiap hari.
- `cin >> hadir;` digunakan untuk menerima input dari pengguna, yaitu 1 untuk hadir dan 0 untuk tidak hadir.
- `totalHadir += hadir;` digunakan untuk menjumlahkan seluruh kehadiran mahasiswa selama 5 hari.
- `int persentase = (totalHadir * 100) / 5;` digunakan untuk menghitung persentase kehadiran berdasarkan jumlah hari mahasiswa hadir.
- Rumus persentase dalam program dapat ditulis sebagai $(\text{jumlah kehadiran} \times 100) \div 5$.
- `cout << "Persentase Kehadiran: " << persentase << "%";` digunakan untuk menampilkan hasil persentase kehadiran kepada pengguna.
- `if (persentase > 75) { }` digunakan untuk menentukan apakah mahasiswa mendapatkan status Kehadiran Baik. Jika persentase lebih dari 75%, maka statusnya Baik.
- `else if (persentase >= 50) { }` digunakan untuk menentukan status Kehadiran Cukup apabila persentase berada di antara 50% sampai 75%.
- `else { }` digunakan apabila kedua kondisi sebelumnya tidak terpenuhi, sehingga mahasiswa mendapatkan status Kehadiran Kurang karena persentasenya kurang dari 50%.
- Keluaran status dapat dilihat pengguna melalui:

```
if (persentase >= 75) {
    cout << "Status Kehadiran: Baik" << endl;
}
else if (persentase >= 50) {
    cout << "Status Kehadiran: Cukup" << endl;
}
else if (persentase <= 50) {
    cout << "Status Kehadiran: Kurang" << endl;
}
```

- `cout << "Ingin mengecek kehadiran untuk mahasiswa lain?"`; digunakan untuk menanyakan kepada pengguna apakah ingin melakukan pengecekan mahasiswa berikutnya.
- `cin >> lanjut`; digunakan untuk menerima jawaban pengguna. Jika memasukkan 1, program akan melakukan pengecekan mahasiswa lain.
- `while (lanjut == 1)` menjadi syarat perulangan agar proses pengecekan mahasiswa dapat dilakukan kembali.
- Jika pengguna memasukkan angka selain 1, kondisi `lanjut == 1` menjadi salah sehingga program akan berhenti melakukan perulangan.
- `return 0`; digunakan sebagai penutup fungsi `main()` dan menandakan bahwa program telah selesai dijalankan dengan normal.

2) Output

```
Apakah mahasiswa hadir di hari ke-1? (1 untuk hadir, 0 untuk tidak hadir): 1
Apakah mahasiswa hadir di hari ke-2? (1 untuk hadir, 0 untuk tidak hadir): 0
Apakah mahasiswa hadir di hari ke-3? (1 untuk hadir, 0 untuk tidak hadir): 0
Apakah mahasiswa hadir di hari ke-4? (1 untuk hadir, 0 untuk tidak hadir): 1
Apakah mahasiswa hadir di hari ke-5? (1 untuk hadir, 0 untuk tidak hadir): 0
Persentase Kehadiran: 40%
Status Kehadiran: Kurang
```

2.2.3 Studi kasus 3 - PEMBAYARAN TAGIHAN LISTRIK

1) pemrograman

```
#include <iostream>
#include <iomanip>
using namespace std;
```

```
int main () {
    float kwh;
    float tarifbln;
    float diskon;
    float trfdiskon;
    int tagihan_tambahan;

    do {
        cout << "Masukkan penggunaan listrik (kwh): ";
        cin >> kwh;

        if (kwh > 0 && kwh <= 100) {
            tarifbln = kwh * 1500;
        }

        if (kwh > 100 && kwh <= 300) {
            tarifbln = kwh * 2000;
        }

        if (kwh > 300) {
            tarifbln = kwh * 3000;
        }

        diskon = 0;

        if (tarifbln > 1000000) {
            diskon = tarifbln * 0.10;
        }

        trfdiskon = tarifbln - diskon;

        cout << fixed << setprecision(2);

        if (kwh > 0) {
            cout << "Total Penggunaan Listrik: "
                 << kwh << " kWh" << endl;

            cout << "Total Tagihan Sebelum Diskon: Rp "
                 << tarifbln << endl;
```

```

        cout << "Diskon: Rp "
              << diskon << endl;

        cout << "Total Tagihan Setelah Diskon: Rp "
              << trfdiskon << endl;
    }

    cout << "Ingin menghitung tagihan untuk penggunaan lain? ";
    cout << "(1 untuk ya, selain itu untuk tidak): ";
    cin >> tagihan_tambahan;

    } while (tagihan_tambahan == 1);

    return 0;
}

```

- **float kwh, tarifbln, diskon, trfdiskon:** Variabel digunakan untuk menyimpan jumlah penggunaan listrik, total tagihan sebelum diskon, jumlah diskon, dan total tagihan setelah diskon.
- **do { ... } while (tagihan_tambahan == 1);**: Struktur *do-while* digunakan agar program dapat menghitung tagihan listrik lebih dari satu kali. Jika pengguna memasukkan angka **1**, proses perhitungan akan diulang dari awal.
- **if (kwh > 0 && kwh <= 100):** Mengecek apakah penggunaan listrik berada antara 1 sampai 100 kWh. Jika memenuhi, tarif yang digunakan adalah Rp1.500 per kWh.
- **if (kwh > 100 && kwh <= 300):** Mengecek apakah penggunaan listrik lebih dari 100 sampai 300 kWh. Jika memenuhi, tarif yang digunakan adalah Rp2.000 per kWh.
- **if (kwh > 300):** Mengecek apakah penggunaan listrik lebih dari 300 kWh. Jika memenuhi, tarif yang digunakan adalah Rp3.000 per kWh.
- **tarifbln = kwh * tarif:** Menghitung total tagihan listrik dengan mengalikan jumlah penggunaan listrik dengan tarif yang sesuai berdasarkan kategori penggunaan kWh.
- **diskon = 0:** Menetapkan nilai awal diskon sebesar Rp0. Jika tagihan tidak memenuhi syarat diskon, maka pengguna tidak mendapatkan potongan.

- `if (tarifbln > 1000000)`: Mengecek apakah total tagihan sebelum diskon lebih dari Rp1.000.000. Jika iya, pengguna mendapatkan diskon sebesar 10%.
- `diskon = tarifbln * 0.10`: Menghitung jumlah diskon sebesar 10% dari total tagihan sebelum diskon.
- `trfdiskon = tarifbln - diskon`: Menghitung total tagihan setelah diskon dengan mengurangi total tagihan sebelum diskon dengan jumlah diskon.
- `fixed << setprecision(2)`: Mengatur format keluaran angka desimal agar selalu menampilkan 2 angka di belakang koma.
- `if (kwh > 0)`: Memastikan hasil perhitungan hanya ditampilkan jika jumlah penggunaan listrik yang dimasukkan lebih dari 0 kWh.
- `cout`: Menampilkan total penggunaan listrik, total tagihan sebelum diskon, jumlah diskon yang diberikan, dan total tagihan setelah diskon kepada pengguna.
- `cin >> tagihan_tambahan`: Menerima pilihan pengguna apakah ingin menghitung tagihan listrik untuk penggunaan lain.
- `while (tagihan_tambahan == 1)`: Jika pengguna memasukkan 1, program akan kembali melakukan perhitungan tagihan listrik. Jika memasukkan angka selain 1, program akan berhenti.

2) Output

```
Masukkan penggunaan listrik (kwh): 500
Total Penggunaan Listrik: 500.00 kwh
Total Tagihan Sebelum Diskon: Rp 1500000.00
Diskon: Rp 150000.00
Total Tagihan Setelah Diskon: Rp 1350000.00
Ingin menghitung tagihan untuk penggunaan lain? (1 untuk ya, selain itu untuk tidak): 1
Masukkan penggunaan listrik (kwh): 90
Total Penggunaan Listrik: 90.00 kwh
Total Tagihan Sebelum Diskon: Rp 135000.00
Diskon: Rp 0.00
Total Tagihan Setelah Diskon: Rp 135000.00
Ingin menghitung tagihan untuk penggunaan lain? (1 untuk ya, selain itu untuk tidak):
```

2.2.4 STUDI KASUS 4 - RATA-RATA NILAI DAN KATEGORI PRESTASI

1) program

```

#include <iostream>
#include <iomanip>

using namespace std;
int main(){
    int mapel;

    cout << fixed << setprecision(2);

    do {
        int nilai_mapel;

        do {
            cout << "masukkan jumlah mata pelajaran: ";
            cin >> nilai_mapel;

            if (nilai_mapel <= 2) {
                cout << "[Peringatan] Jumlah pembelian harus minimal 3! Silakan coba lagi.\n\n";
            }
        } while (nilai_mapel <= 2);

        double total_nilai = 0;
        for (int i = 1; i <= nilai_mapel; i++) {
            double minimal_nilai;
            cout << "Masukkan nilai mata pelajaran ke-" << i << ": ";
            cin >> minimal_nilai;
            total_nilai += minimal_nilai;
        }
        double rata_rata = total_nilai / nilai_mapel;

        string prestasi = "";
        if (rata_rata >= 85){
            total_nilai += minimal_nilai;
        }
        double rata_rata = total_nilai / nilai_mapel;

        string prestasi = "";
        if (rata_rata >= 85){
            prestasi = "sangat baik";
        } else if (rata_rata >= 70){
            prestasi = "baik";
        } else if (rata_rata >= 50){
            prestasi = "cukup";
        } else {
            prestasi = "perlu peningkatan";
        }

        cout << "nilai rata-rata: " << rata_rata << endl;
        cout << "prestasi: " << prestasi << endl;

        cout << "ingin menghitung nilai siswa lain? (1 untuk ya, selain itu tidak): ";
        cin >> mapel;
        cout << endl;

    } while (mapel == 1);

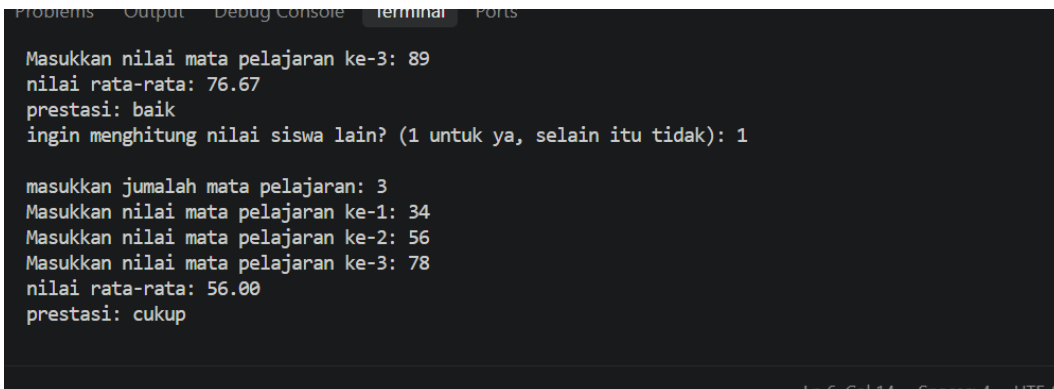
    return 0;
}

```

- `#include <iostream>` & `#include <iomanip>`: Pustaka untuk menangani input-output (`cin`, `cout`) dan pembentukan format tampilan angka (`setprecision`).
- `using namespace std;`: Mempermudah pemanggilan fungsi standar C++ tanpa menuliskan perintah `std::` secara berulang.
- `fixed << setprecision(2)`: Mengatur format keluaran angka desimal agar selalu menampilkan 2 angka di belakang koma.
- `do { ... } while (mapel == 1);`: Struktur *do-while* paling luar ini berfungsi agar program dapat menghitung nilai beberapa siswa berturut-turut. Di akhir program, jika pengguna memasukkan angka 1, seluruh proses perhitungan akan diulang dari awal.

- `do { ... } while (nilai_mapel <= 2);`: Struktur *do-while* kedua ini bertindak sebagai validator input. Program mewajibkan pengguna memasukkan jumlah mata pelajaran lebih dari 2 (minimal 3). Jika pengguna memasukkan angka kurang dari atau sama dengan 2, pesan peringatan akan muncul dan program menolak melanjutkan ke proses berikutnya hingga syarat terpenuhi.
- `for (int i = 1; i <= nilai_mapel; i++)`: Perulangan `for` digunakan untuk meminta input nilai setiap mata pelajaran satu per satu sesuai jumlah yang diinputkan sebelumnya.
- `total_nilai += minimal_nilai;` Setiap nilai yang diinputkan akan langsung dijumlahkan ke dalam variabel `total_nilai`.
- `double rata_rata = total_nilai / nilai_mapel;` Menghitung rata-rata dengan membagi akumulasi seluruh nilai dengan total mata pelajaran.
- `for (int i = 1; i <= nilai_mapel; i++)`: Perulangan `for` digunakan untuk meminta input nilai setiap mata pelajaran satu per satu sesuai jumlah yang diinputkan sebelumnya.
- `total_nilai += minimal_nilai;` Setiap nilai yang diinputkan akan langsung dijumlahkan ke dalam variabel `total_nilai`.
- `double rata_rata = total_nilai / nilai_mapel;` Menghitung rata-rata dengan membagi akumulasi seluruh nilai dengan total mata pelajaran.

2) Output



```
Problems  Output  Debug Console  Terminal  Ports

Masukkan nilai mata pelajaran ke-3: 89
nilai rata-rata: 76.67
prestasi: baik
ingin menghitung nilai siswa lain? (1 untuk ya, selain itu tidak): 1

masukkan jumlah mata pelajaran: 3
Masukkan nilai mata pelajaran ke-1: 34
Masukkan nilai mata pelajaran ke-2: 56
Masukkan nilai mata pelajaran ke-3: 78
nilai rata-rata: 56.00
prestasi: cukup
```

2.2.5 STUDI KASUS 5 - PENGELOLAAN PENGELUARAN HARIAN

1) pemrograman

```

1  #include <iostream>
2  #include <string>
3  #include <iomanip>
4  #include <algorithm>
5
6  using namespace std;
7
8
9  string toLower(string str) {
10     transform(str.begin(), str.end(), str.begin(), ::tolower);
11     return str;
12 }
13
14 int main() {
15     int ulang = 1;
16
17     do {
18         double totalMakanan = 0;
19         double totalTransportasi = 0;
20         double totalHiburan = 0;
21         double totalLainnya = 0;
22
23         double pengeluaranTerbesar = -1;
24         string kategoriPengeluaranTerbesar = "";
25
26         for (int i = 1; i <= 7; ++i) {
27             string kategori;
28             double jumlah;
29
30             cout << "Masukkan kategori pengeluaran hari ke-" << i << " (Makanan/Transportasi/Hiburan/Lain-lain) : ";
31             cin >> kategori;
32

```

```

7         do {
8             for (int i = 1; i <= 7; ++i) {
9                 cout << "Masukkan kategori pengeluaran hari ke-" << i << " (Makanan/Transportasi/Hiburan/Lain-lain) : ";
10                 cin >> kategori;
11
12                 cout << "Masukkan jumlah pengeluaran: Rp ";
13                 cin >> jumlah;
14
15                 string katLower = toLower(kategori);
16
17                 if (katLower == "makanan") {
18                     totalMakanan += jumlah;
19                 } else if (katLower == "transportasi") {
20                     totalTransportasi += jumlah;
21                 } else if (katLower == "hiburan") {
22                     totalHiburan += jumlah;
23                 } else {
24                     totalLainnya += jumlah;
25                 }
26
27                 if (jumlah > pengeluaranTerbesar) {
28                     pengeluaranTerbesar = jumlah;
29                     kategoriPengeluaranTerbesar = kategori;
30                 }
31             }
32
33             double totalSeminggu = totalMakanan + totalTransportasi + totalHiburan + totalLainnya;
34

```

```

14 int main() {
15     do {
26         for (int i = 1; i <= 7; ++i) {
50             if (jumlah > pengeluaranTerbesar) {
51                 pengeluaranTerbesar = jumlah;
52                 kategoriPengeluaranTerbesar = kategori;
53             }
54         }
55
56         double totalSeminggu = totalMakanan + totalTransportasi + totalHiburan + totalLainnya;
57
58
59         cout << fixed << setprecision(2);
60         cout << "Total Pengeluaran Makanan: Rp " << totalMakanan << endl;
61         cout << "Total Pengeluaran Transportasi: Rp " << totalTransportasi << endl;
62         cout << "Total Pengeluaran Hiburan: Rp " << totalHiburan << endl;
63         cout << "Total Pengeluaran Lainnya: Rp " << totalLainnya << endl;
64         cout << "Total Pengeluaran Selama Seminggu: Rp " << totalSeminggu << endl;
65         cout << "Pengeluaran Terbesar: Rp " << pengeluaranTerbesar << " pada kategori " << kategoriPengeluaranTerbesar << endl;
66
67         cout << "Ingin mencatat pengeluaran untuk minggu lain? (1 untuk ya, selain itu untuk tidak): ";
68         cin >> ulang;
69
70         cout << endl;
71
72     } while (ulang == 1);
73
74     return 0;
75 }

```

- `do { ... } while (ulang == 1);`: Struktur *do-while* paling luar ini berfungsi agar program dapat mengulang proses pencatatan pengeluaran harian untuk minggu-minggu berikutnya jika pengguna memasukkan angka 1 di akhir program.
- `double totalMakanan = 0, totalTransportasi = 0, totalHiburan = 0, totalLainnya = 0;`: Deklarasi variabel pecahan (*double*) untuk menampung dan menjumlahkan total nominal pengeluaran dari masing-masing kategori, yang diawali dari angka 0 pada setiap minggunya.
- `double pengeluaranTerbesar = -1;`: Variabel pembanding untuk mencatat nilai pengeluaran tertinggi harian, diinisialisasi dengan angka negatif agar input pertama yang bernilai 0 atau lebih selalu berhasil menjadi rekor terbesar awal.
- `string kategoriPengeluaranTerbesar = "";`: Variabel teks (*string*) yang digunakan untuk menyimpan nama kategori dari pengeluaran harian terbesar yang ditemukan.
- `for (int i = 1; i <= 7; ++i)`: Perulangan *for* yang berjalan sebanyak 7 kali untuk meminta input kategori dan nominal pengeluaran harian secara berturut-turut dari Hari ke-1 sampai Hari ke-7.
- `cin >> kategori;` & `cin >> jumlah;`: Perintah *input* untuk menerima masukan nama kategori dan nominal pengeluaran harian dari pengguna melalui keyboard.
- `string katLower = toLower(kategori);`: Memanggil fungsi kustom `toLower` untuk mengubah seluruh huruf pada string kategori menjadi huruf kecil, sehingga input tidak bersifat sensitif terhadap huruf kapital (*case-insensitive*).
- `if (katLower == "makanan") ... else:` Struktur percabangan *if-else-if* yang bertugas mengidentifikasi kategori input dan menambahkan nominal

- pengeluaran ke variabel total yang sesuai (Makanan, Transportasi, Hiburan, atau ke `totalLainnya` jika di luar tiga kategori utama).
- `if (jumlah > pengeluaranTerbesar)`: Logika kondisi untuk mengecek apakah pengeluaran pada hari tersebut lebih besar dari rekor sebelumnya; jika ya, nilai `pengeluaranTerbesar` dan `kategoriPengeluaranTerbesar` akan diperbarui.
 - `double totalSeminggu = totalMakanan + totalTransportasi + totalHiburan + totalLainnya`; Menghitung total keseluruhan pengeluaran selama seminggu dengan menjumlahkan akumulasi nilai dari keempat kategori.
 - `cout << fixed << setprecision(2)`; Mengatur format cetak angka desimal agar selalu menampilkan tepat dua angka di belakang koma (misalnya `Rp 200000.00`) sesuai tampilan luaran.
 - `cout << "Total Pengeluaran ..."`; Perintah *output* untuk mencetak rincian ringkasan total pengeluaran tiap kategori, total seminggu, serta nilai dan kategori pengeluaran terbesar ke layar terminal.
 - `cin >> ulang`; Menerima input pilihan pengguna (angka 1 untuk mengulang pencatatan minggu baru, atau angka selain 1 untuk menghentikan program).

2) Output

```

Problems  Output  Debug Console  Terminal  Ports
Masukkan jumlah pengeluaran: Rp 20000
Masukkan kategori pengeluaran hari ke-6 (Makanan/Transportasi/Hiburan/Lain-lain): lain-lain
Masukkan jumlah pengeluaran: Rp 34000
Masukkan kategori pengeluaran hari ke-7 (Makanan/Transportasi/Hiburan/Lain-lain): hiburan
Masukkan jumlah pengeluaran: Rp 60000
Total Pengeluaran Makanan: Rp 90000.00
Total Pengeluaran Transportasi: Rp 980000.00
Total Pengeluaran Hiburan: Rp 60000.00
Total Pengeluaran Lainnya: Rp 154000.00
Total Pengeluaran Selama Seminggu: Rp 1284000.00
Pengeluaran Terbesar: Rp 980000.00 pada kategori transportasi
Ingin mencatat pengeluaran untuk minggu lain? (1 untuk ya, selain itu untuk tidak): 

```

BAB III KESIMPULAN

3.1 Kesimpulan

Berdasarkan materi looping pada bahasa pemrograman C++, dapat disimpulkan bahwa perulangan adalah struktur kontrol yang menjalankan sekumpulan pernyataan berulang kali selama suatu kondisi terpenuhi. Dengan perulangan, program menjadi lebih singkat, efisien, dan mudah dikelola karena instruksi yang sama tidak perlu ditulis berulang.

Pernyataan `while` menguji kondisi di awal sehingga cocok untuk perulangan yang jumlahnya belum pasti, seperti menanyakan "ingin menambah belanjaan lagi?" pada studi kasus harga dan diskon. Pernyataan `for` lebih tepat ketika jumlah pengulangan sudah diketahui, seperti 5 hari kehadiran pada studi kasus absensi dan 7 hari pencatatan pada studi kasus pengeluaran harian.

Melalui 10 studi kasus (harga dan diskon barang, absensi mahasiswa, tagihan listrik, rata-rata nilai, dan pengelolaan pengeluaran harian), terlihat bahwa perulangan hampir selalu bekerja bersama percabangan (`if-else`). Perulangan mengurus pengambilan dan pengolahan data berulang, sedangkan percabangan menentukan keputusan, seperti besar diskon, tarif listrik, status kehadiran, atau kategori prestasi. Pemilihan jenis perulangan yang tepat, penulisan kondisi yang benar, dan pengujian dengan berbagai nilai input penting agar program berjalan sesuai harapan dan tidak terjebak dalam perulangan tanpa henti (`infinite loop`).

LAMPIRAN

Link Github

1. Nadhira

<https://github.com/AkebiAmono/tugas-algoritma-nadh>

2. Anandhika Rizaldy

<https://github.com/AnandhikaR>

3. Ahmad Ibrohim Shofiyullah

<https://github.com/ahmadibrohim2605356-ops/praktikum-4>

4. Bintang Rachmad Hidayat

[https://github.com/bintangrachmadh-spec/ALGORITMA_DAN_PEMOGRAMAN_TI/tree/main/SEMESTER%20GANJIL/S1/P01%20-%20Copy%20\(4\)](https://github.com/bintangrachmadh-spec/ALGORITMA_DAN_PEMOGRAMAN_TI/tree/main/SEMESTER%20GANJIL/S1/P01%20-%20Copy%20(4))

5. Muhammad Zaki Aliyyudin

<https://github.com/muhammadzaki2605356>

6. Nadila Zahra' Dzakiyyah

<https://github.com/arla-alesha>